

Accenture Frontend Assessment

Practice question bank: HTML + CSS + JavaScript

19 questions | easy to hard | questions only, no solutions

How the exam is shaped

You are given a mostly finished HTML page and CSS file. One or two very small HTML / CSS tweaks are asked for (Task A), but almost all of the marks are for the JavaScript that makes the page work (Task B). Every question below follows that same shape.

How to practise

- Create one `.html` file per question. Paste the given HTML inside `<body>`, the CSS inside `<style>`, and write your JavaScript in a `<script>` tag at the end of the body.
- Use plain browser JavaScript only: `getElementById`, `querySelector(All)`, `addEventListener`, `classList`, `dataset`, `textContent` / `innerHTML`, `createElement` / `appendChild`, `localStorage`, `setTimeout`. No libraries or frameworks.
- Do the tasks in order. Every bullet in Task B is a separate requirement that can be checked on its own, so test each one, including empty input, repeated clicks and whitespace.
- Target times are for the whole question. If you go past 1.5 times the target, note the topic and revisit it.
- Solve without looking anything up first. Look up only what you were truly stuck on.

Roadmap

#	Question	Level	Main concepts	Time
1	Theme Toggle	Easy	click event, <code>classList.toggle</code> , <code>textContent</code>	5 min
2	Country Dropdown	Easy	change event, <code>select.value</code> , object lookup	8 min
3	Bio Box with Characters Left	Easy	input event, string length, conditional classes, disabled attribute	8 min
4	Planet Card Swapper	Easy	helper functions, updating several elements, active button state	10 min
5	Login Form Validation	Easy	submit event, <code>preventDefault</code> , early return, showing messages	10 min
6	Inventory Table - Stock Highlighting	Easy-Medium	<code>querySelectorAll</code> , table cells, <code>Number()</code> , <code>classList</code> , counting	12 min
7	Promo Modal	Easy-Medium	<code>classList</code> , <code>event.target</code> , <code>setTimeout</code> , <code>keydown</code> event	15 min
8	Tab Switcher	Easy-Medium	dataset, matching buttons to panels, <code>classList.toggle(name, condition)</code>	12 min
9	FAQ Accordion	Easy-Medium	<code>querySelectorAll</code> , <code>nextElementSibling</code> , closing siblings, state before change	15 min
10	Star Rating with Label and Reset	Easy-Medium	event delegation, dataset, <code>classList.toggle(name, condition)</code> , arrays	15 min
11	Password Strength and Confirmation	Medium	input events, multiple validation states, click handler with ordered checks	20 min
12	To-Do List: Add, Complete, Delete	Medium	<code>createElement</code> / <code>appendChild</code> , event delegation, <code>classList.toggle</code> , counters	20 min
13	Shopping Cart with Decimal Prices	Medium	event delegation, data attributes, <code>Number()</code> , <code>toFixed(2)</code> , resetting state	20 min
14	Product Grid: Search + Category Filter	Medium	input + change events, combining two filters, dataset, empty state	25 min
15	Student Marks Table with Filter	Medium	<code>createElement</code> / <code>innerHTML</code> , validation, filter that survives new rows, summary line	25 min
16	To-Do List with <code>localStorage</code> and Filters	Hard	<code>localStorage</code> + JSON, rendering from an array, filter views, index bugs	30 min
17	Cart with Quantity Controls and Coupons	Hard	delegation, <code>closest()</code> , recalculation, conditional discounts, empty state	35 min
18	Employee Table: Search, Filter and Sort	Hard	combined filters, numeric vs text sorting, toggling sort direction, visible totals	35 min
19	Registration Form with Real-Time Validation	Hard	input / blur / change events, per-field rules, touched state, enabling submit	40 min

Total target time: about 6 hours 0 minutes.

LEVEL 1 - WARM-UP

Questions 1 - 5 | about 41 minutes in total

One event, one or two DOM updates. If you can do these without looking anything up, you have the basics of `getElementById`, `addEventListener`, `classList` and `textContent`.

Q1 Theme Toggle

Easy

Target time: 5 min

Concepts: *click event*, *classList.toggle*, *textContent*

A page has a single button that switches between a light and a dark theme.

GIVEN HTML

```
<h2>Welcome to my page</h2>
<p>Click the button to switch the theme.</p>
<button id="theme-btn">Switch to Dark</button>
```

GIVEN CSS

```
body { font-family: Arial, sans-serif; padding: 20px; background: #ffffff; color: #222; }
body.dark { background: #1e1e2e; color: #f5f5f5; }
button { padding: 8px 16px; cursor: pointer; }
```

Task A - HTML / CSS tweak

In the CSS, give `#theme-btn` a blue background (`#2563eb`), white text and `border-radius: 8px`.

Task B - JavaScript

- Add a click listener to `#theme-btn`.
- Each click toggles the class `dark` on the `body` element.
- While the dark theme is on, the button text must read "Switch to Light"; while it is off, it must read "Switch to Dark".

Q2 Country Dropdown

Easy

Target time: 8 min

Concepts: *change event*, *select.value*, *object lookup*

Choosing a country from a dropdown should immediately show its capital and currency.

GIVEN HTML

```
<select id="country-select">
  <option value="">-- Select a country --</option>
  <option value="india">India</option>
  <option value="japan">Japan</option>
  <option value="brazil">Brazil</option>
</select>
<div id="country-info"></div>
```

GIVEN CSS

```
select { padding: 8px; font-size: 15px; }
#country-info { margin-top: 12px; font-size: 15px; }
```

Task A - HTML / CSS tweak

Give `#country-info` a light grey background (`#f1f1f1`) and `10px` padding.

Task B - JavaScript

- When the selection changes, look up the chosen country's capital and currency: India = New Delhi / Rupee, Japan = Tokyo / Yen, Brazil = Brasilia / Real.
- Show them inside `#country-info` as `Capital: ... | Currency: ....`
- If the placeholder option is chosen, clear `#country-info`.
- Keep the data in one object. Do not write an if / else-if chain per country.

Q3 Bio Box with Characters Left

Easy
Target time: 8 min

Concepts: *input event, string length, conditional classes, disabled attribute*

A short-bio textarea shows how many characters are still available and only lets the user save when something is typed.

GIVEN HTML

```
<textarea id="bio" maxlength="80" placeholder="Write a short bio..."></textarea>
<p>Characters left: <span id="left">80</span></p>
<button id="save-btn" disabled>Save</button>
```

GIVEN CSS

```
textarea { width: 100%; height: 80px; padding: 8px; }
#left.warning { color: #d97706; font-weight: bold; }
#left.empty { color: #c0392b; font-weight: bold; }
button:disabled { opacity: 0.5; cursor: not-allowed; }
```

Task A - HTML / CSS tweak

Stop the textarea from being resizable and give it a **2px solid #2563eb** border.

Task B - JavaScript

- On every **input** event, set **#left** to 80 minus the current number of characters.
- When 10 or fewer characters are left, the counter has the class **warning**. When 0 are left, it has the class **empty** instead. Above 10 it has neither.
- #save-btn** is disabled while the textarea is empty and enabled as soon as there is at least one character.

Q4 Planet Card Swapper

Easy
Target time: 10 min

Concepts: *helper functions, updating several elements, active button state*

Three buttons pick a planet. Clicking one updates a shared image, name and fact.

GIVEN HTML

```
<div class="picker">
  <button id="btn-mars">Mars</button>
  <button id="btn-venus">Venus</button>
  <button id="btn-jupiter">Jupiter</button>
</div>
<img id="planet-image" src="" alt="Selected planet">
<h3 id="planet-name">Pick a planet</h3>
<p id="planet-fact"></p>
```

GIVEN CSS

```
.picker button { margin-right: 10px; padding: 8px 16px; cursor: pointer; }
.picker button.active { background: #2c3e50; color: #fff; }
#planet-image { width: 160px; height: 160px; object-fit: cover; display: block;
margin-top: 12px; }
```

Task A - HTML / CSS tweak

Make **#planet-image** a circle (**border-radius: 50%**).

Task B - JavaScript

- Write ONE helper function **showPlanet(src, name, fact)** that updates the image **src**, the name and the fact. Both other handlers must call it; no duplicated update code.
- Mars: **mars.jpg**, "Mars", "Olympus Mons on Mars is the tallest volcano in the solar system."
- Venus: **venus.jpg**, "Venus", "A day on Venus is longer than its year."
- Jupiter: **jupiter.jpg**, "Jupiter", "Jupiter's Great Red Spot is a storm larger than Earth."
- The clicked button gets the class **active**; the other two lose it.

Q5 Login Form Validation

Easy
Target time: 10 min

Concepts: *submit event, preventDefault, early return, showing messages*

A login form must not submit if the input is invalid. It should show an inline message and keep the user on the page.

GIVEN HTML

```
<form id="login-form">
  <label>Email: <input type="text" id="email"></label><br><br>
  <label>Password: <input type="password" id="password"></label><br><br>
  <p id="msg" class="hidden"></p>
  <button type="submit">Log In</button>
</form>
```

GIVEN CSS

```
.hidden { display: none; }
#msg.error { color: #c0392b; font-size: 14px; }
#msg.success { color: #1e8449; font-size: 14px; }
input { padding: 6px; margin: 4px 0; }
```

Task A - HTML / CSS tweak

Make the Log In button take the full width of its container (**width: 100%**).

Task B - JavaScript

- Listen for the form's **submit** event and stop the default submission.
- Check in this order and stop at the first failure: (1) email blank (after trimming) -> "Email is required"; (2) email has no @ -> "Enter a valid email"; (3) password shorter than 6 characters -> "Password must be at least 6 characters".
- A failure shows the message in **#msg** with the class **error**.
- If everything is valid, show "Logged in successfully!" with the class **success** (and without **error**), and clear both inputs.

LEVEL 2 - BUILDING BLOCKS

Questions 6 - 10 | about 69 minutes in total

Loops over groups of elements, class toggling with a condition, and deciding what to do from the element that was clicked.

Q6 Inventory Table - Stock Highlighting

Easy-Medium
Target time: 12 min

Concepts: `querySelectorAll`, `table cells`, `Number()`, `classList`, `counting`

An inventory table is already filled in. On page load, rows must be styled according to their stock level. No button is involved.

GIVEN HTML

```
<table id="inventory">
  <thead><tr><th>Product</th><th>Price</th><th>Stock</th></tr></thead>
  <tbody>
    <tr><td>Keyboard</td><td>$25</td><td>12</td></tr>
    <tr><td>Mouse</td><td>$15</td><td>0</td></tr>
    <tr><td>Monitor</td><td>$120</td><td>3</td></tr>
    <tr><td>Webcam</td><td>$40</td><td>0</td></tr>
    <tr><td>Headset</td><td>$60</td><td>5</td></tr>
  </tbody>
</table>
<p id="summary"></p>
```

GIVEN CSS

```
table { border-collapse: collapse; width: 100%; }
td, th { border: 1px solid #ccc; padding: 8px 12px; }
.out-of-stock { background-color: #fdecea; }
.low-stock { background-color: #fff4e0; }
```

Task A - HTML / CSS tweak

Make the table header cells dark blue (**#1e3a8a**) with white text.

Task B - JavaScript

- Select every row in the table body and read the number in its third cell.
- Stock of 0: add the class **out-of-stock** to the row and change that cell's text to "Out of Stock".
- Stock from 1 to 5 (inclusive): add the class **low-stock** to the row.
- Write **X out of stock, Y low on stock** into **#summary** (for the data above: "2 out of stock, 2 low on stock").

Q7 Promo Modal

Easy-Medium
Target time: 15 min

Concepts: `classList`, `event.target`, `setTimeout`, `keydown event`

A promo pop-up can be opened with a button, closed in several ways, and also opens by itself after a delay.

GIVEN HTML

```
<button id="open-btn">Open Promo Modal</button>

<div id="promo-modal" class="modal-overlay hidden">
  <div class="modal-content">
    <h3>Exclusive Offer!</h3>
    <p>Get 20% off your purchase!</p>
    <button id="close-btn">Close</button>
  </div>
</div>
```

GIVEN CSS

```
.modal-overlay { position: fixed; inset: 0; background: rgba(0, 0, 0, 0.7);
                  display: grid; place-items: center; }
.modal-content { background: #1e293b; color: #fff; padding: 20px; border-radius: 8px;
                  text-align: center; }
.hidden { display: none !important; }
```

Task A - HTML / CSS tweak

Change the modal box background to `#0f766e` and make the heading text gold.

Task B - JavaScript

- `#open-btn` shows the modal; `#close-btn` hides it (by removing / adding `hidden`).
- Clicking the dark backdrop closes the modal, but clicking inside the box itself must NOT close it.
- Pressing the Escape key closes the modal.
- The modal opens automatically 4 seconds after the page loads.
- Bonus: skip the automatic opening if the user has already opened the modal manually before the 4 seconds are up.

Q8 Tab Switcher

Easy-Medium
Target time: 12 min

Concepts: dataset, matching buttons to panels, `classList.toggle(name, condition)`

Three tab buttons control three content panels. Exactly one tab and one panel must be active at any time.

GIVEN HTML

```
<div class="tabs">
  <button class="tab-btn active" data-tab="overview">Overview</button>
  <button class="tab-btn" data-tab="specs">Specs</button>
  <button class="tab-btn" data-tab="reviews">Reviews</button>
</div>
<div class="tab-panel" id="overview">Overview content goes here.</div>
<div class="tab-panel hidden" id="specs">Specs content goes here.</div>
<div class="tab-panel hidden" id="reviews">Reviews content goes here.</div>
<p id="current">Current tab: overview</p>
```

GIVEN CSS

```
.hidden { display: none; }
.tab-btn { padding: 8px 16px; cursor: pointer; border: 1px solid #ccc;
            background: #f5f5f5; }
.tab-btn.active { background: #2c3e50; color: #fff; }
.tab-panel { padding: 16px; border: 1px solid #eee; }
```

Task A - HTML / CSS tweak

Make the tab buttons have rounded top corners (`border-radius: 8px 8px 0 0`).

Task B - JavaScript

- Clicking a tab shows the panel whose `id` equals the button's `data-tab` value and hides the other panels.
- The clicked button gets `active`; the others lose it.
- `#current` always reads `Current tab: <name>` using the clicked tab's `data-tab` value.
- Do not hardcode the number of tabs. The same code must work if a fourth tab is added to the HTML.

Q9 FAQ Accordion

Easy-Medium
Target time: 15 min

Concepts: *querySelectorAll*, *nextElementSibling*, *closing siblings*, *state before change*

Three FAQ questions each expand an answer on click. Only one answer may be open at a time, and each question shows a + or - icon.

GIVEN HTML

```
<div class="faq-item">
  <div class="faq-question"><span class="icon">+</span> What is your return policy?</div>
  <div class="faq-answer hidden">Items can be returned within 30 days.</div>
</div>
<div class="faq-item">
  <div class="faq-question"><span class="icon">+</span> Do you ship internationally?</div>
  <div class="faq-answer hidden">Yes, to over 40 countries.</div>
</div>
<div class="faq-item">
  <div class="faq-question"><span class="icon">+</span> How do I track my order?</div>
  <div class="faq-answer hidden">Use the tracking link in your confirmation email.</div>
</div>
```

GIVEN CSS

```
.hidden { display: none; }
.faq-question { cursor: pointer; padding: 10px; background: #f0f0f0; font-weight: bold; }
.faq-answer { padding: 10px; border: 1px solid #eee; }
.icon { display: inline-block; width: 18px; }
```

Task A - HTML / CSS tweak

Give `.faq-question` a `1px solid #ddd` bottom border and change its background to `#e8f0fe` on hover.

Task B - JavaScript

- Clicking a question opens its own answer.
- Opening one answer automatically closes any other open answer.
- Clicking the question that is already open closes it again.
- The `.icon` of the open question shows "-"; every other question's icon shows "+".

Q10 Star Rating with Label and Reset

Easy-Medium
Target time: 15 min

Concepts: *event delegation*, *dataset*, *classList.toggle(name, condition)*, *arrays*

Five stars let a user choose a rating from 1 to 5. The chosen rating is shown as a number and as a word.

GIVEN HTML

```
<div id="stars">
  <span class="star" data-value="1">&#9734;</span>
  <span class="star" data-value="2">&#9734;</span>
  <span class="star" data-value="3">&#9734;</span>
  <span class="star" data-value="4">&#9734;</span>
  <span class="star" data-value="5">&#9734;</span>
</div>
<p>Rating: <span id="rating-value">0</span>/5 - <span id="rating-text"></span></p>
<button id="reset-btn">Reset</button>
```

GIVEN CSS

```
#stars { font-size: 28px; cursor: pointer; }  
.star { color: #ccc; }  
.star.filled { color: gold; }
```

Task A - HTML / CSS tweak

Make the Reset button red (**#c0392b**) with white text.

Task B - JavaScript

- Use one click listener on **#stars** (event delegation).
- Clicking a star with value N gives the class **filled** to every star with value $\leq N$ and removes it from the rest.
- **#rating-value** shows N and **#rating-text** shows the matching word: 1 Poor, 2 Fair, 3 Good, 4 Very good, 5 Excellent.
- Clicking the space between stars (inside **#stars** but not on a star) must do nothing.
- **#reset-btn** sets the rating back to 0, unfills all stars and empties **#rating-text**.

LEVEL 3 - REAL EXAM LEVEL

Questions 11 - 15 | about 110 minutes in total

Several parts of the page must stay in sync: live filters, event delegation, running totals and dynamically created elements. This is the level the Accenture round is set at.

Q11 Password Strength and Confirmation

Medium
Target time: 20 min

Concepts: *input events, multiple validation states, click handler with ordered checks*

A sign-up box shows password strength while typing, live confirmation feedback, and a final check on Submit.

GIVEN HTML

```
<label>Password: <input type="password" id="pass"></label>
<p id="strength"></p>
<label>Confirm: <input type="password" id="con-pass"></label>
<p id="match-msg"></p>
<button id="submit-btn">Submit</button>
<p id="submit-msg"></p>
```

GIVEN CSS

```
.weak { color: #c0392b; }
.medium { color: #d68910; }
.strong { color: #1e8449; }
.ok { color: #1e8449; }
.bad { color: #c0392b; }
input { padding: 6px; margin: 4px 0; }
```

Task A - HTML / CSS tweak

Add the placeholder "Create password" to `#pass` and "Repeat password" to `#con-pass`.

Task B - JavaScript

- Strength meter: on every `input` in `#pass`, write into `#strength` "Weak" (fewer than 4 characters), "Medium" (4 to 7) or "Strong" (8 or more) and set the matching class `weak` / `medium` / `strong`. An empty password shows nothing.
- Live match message: while typing in either field, `#match-msg` shows "Passwords match" (class `ok`) or "Passwords do not match" (class `bad`). Show nothing while the confirm field is empty.
- On Submit, check in this order and stop at the first failure: password empty -> "Password cannot be empty"; passwords differ -> "Failed"; strength is Weak -> "Password too weak". Otherwise show "Success!!!" in `#submit-msg`.

Q12 To-Do List: Add, Complete, Delete

Medium
Target time: 20 min

Concepts: *createElement / appendChild, event delegation, classList.toggle, counters*

A to-do list where tasks can be added, marked done and deleted, with a live counter underneath.

GIVEN HTML

```
<input type="text" id="task-input" placeholder="New task">
<button id="add-btn">Add</button>
<ul id="task-list"></ul>
<p id="counter">0 tasks, 0 done</p>
```

GIVEN CSS

```
.done { text-decoration: line-through; color: #888; }
li { cursor: pointer; padding: 4px 0; }
.remove-btn { margin-left: 10px; color: #c0392b; cursor: pointer; }
```

Task A - HTML / CSS tweak

Add a light grey bottom border (1px solid #eee) to every li.

Task B - JavaScript

- Clicking "Add" with non-empty text creates an `<li>` containing the task text and a `<span class="remove-btn">x</span>`, appends it to `#task-list`, clears the input and puts the cursor back in the input.
- Whitespace-only input is ignored. Pressing Enter inside the input also adds the task.
- Clicking a task's text toggles the class `done` on that `<li>`.
- Clicking the "x" removes that `<li>` and must not toggle `done` on the way.
- Use a single delegated listener on `#task-list` for toggling and removing.
- `#counter` always reads `N tasks, M done` and is updated after every add, toggle and removal.

Q13 Shopping Cart with Decimal Prices

Medium

Target time: 20 min

Concepts: event delegation, data attributes, `Number()`, `toFixed(2)`, resetting state

Each row has an Add button. The page tracks quantities, the number of items and a running total, and has a Clear button.

GIVEN HTML

```
<table id="cart">
  <tr><th>Item</th><th>Price</th><th>Qty</th><th></th></tr>
  <tr><td>Notebook</td><td>$5.50</td><td id="qty-notebook">0</td>
    <td><button class="add-btn" data-item="notebook" data-price="5.50">Add</button></td></tr>
  <tr><td>Pen</td><td>$2.25</td><td id="qty-pen">0</td>
    <td><button class="add-btn" data-item="pen" data-price="2.25">Add</button></td></tr>
  <tr><td>Eraser</td><td>$1.00</td><td id="qty-eraser">0</td>
    <td><button class="add-btn" data-item="eraser" data-price="1.00">Add</button></td></tr>
</table>
<p>Items in cart: <span id="cart-count">0</span></p>
<h3>Total: $<span id="cart-total">0.00</span></h3>
<button id="clear-btn">Clear Cart</button>
```

GIVEN CSS

```
table { border-collapse: collapse; width: 100%; }
td, th { border: 1px solid #ccc; padding: 8px 12px; text-align: left; }
.add-btn { padding: 4px 12px; cursor: pointer; }
```

Task A - HTML / CSS tweak

Change the Add buttons to a green background (`#2e7d32`) with white text.

Task B - JavaScript

- Use one click listener that serves all three Add buttons (delegation on the table).
- Clicking Add increases that item's quantity cell (`#qty-<item>`) by 1 and leaves the other rows alone.
- `#cart-count` shows the total number of items across all rows.
- `#cart-total` always shows exactly two decimals. Example: Notebook, Notebook, Pen -> count 3, total 13.25.
- `#clear-btn` sets every quantity back to 0, the count to 0 and the total to 0.00.

Q14 Product Grid: Search + Category Filter

Medium
Target time: 25 min

Concepts: *input + change events, combining two filters, dataset, empty state*

A product grid can be filtered by a live text search and a category radio button. Both filters apply at the same time.

GIVEN HTML

```
<input type="text" id="p-search" placeholder="Search product...">

<div class="categories">
  <label><input type="radio" name="p-cat" value="all" checked> All</label>
  <label><input type="radio" name="p-cat" value="tech"> Tech</label>
  <label><input type="radio" name="p-cat" value="book"> Books</label>
</div>

<div class="grid">
  <div class="p-card" data-cat="tech">Laptop</div>
  <div class="p-card" data-cat="book">JS Handbook</div>
  <div class="p-card" data-cat="book">CPP Handbook</div>
  <div class="p-card" data-cat="tech">CPU</div>
  <div class="p-card" data-cat="book">HTML Book</div>
  <div class="p-card" data-cat="book">Tech Handbook</div>
  <div class="p-card" data-cat="tech">Camera</div>
</div>
<p id="result-count"></p>
<p id="no-result" class="hidden">No products found</p>
```

GIVEN CSS

```
.grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(140px, 1fr));
        gap: 10px; }
.p-card { border: 1px solid #ccc; padding: 16px; border-radius: 6px; }
.categories label { margin-right: 12px; }
.hidden { display: none; }
```

Task A - HTML / CSS tweak

Add a soft box-shadow to `.p-card` and change its border colour to `#94a3b8`.

Task B - JavaScript

- Filter live as the user types and whenever a radio button changes.
- A card is visible only if its text contains the search text (case-insensitive) AND its `data-cat` matches the selected category (or "All" is selected).
- Hide cards with the class `hidden`, not by editing inline styles.
- `#result-count` shows `Showing X of 7 products`.
- When X is 0, show `#no-result`; otherwise hide it.
- Edge case to test: searching "tech" with the Tech category selected must show 0 products, because "Tech Handbook" is a book.

Q15 Student Marks Table with Filter

Medium
Target time: 25 min

Concepts: `createElement` / `innerHTML`, validation, filter that survives new rows, summary line

Marks are entered for students. Each row shows a pass / fail status (pass mark is 40). A dropdown filters the table.

GIVEN HTML

```
<input type="text" id="name" placeholder="Student name">
<input type="number" id="mark" min="0" max="100" placeholder="Marks">
<button id="add-btn">Add</button>
<p id="error" class="hidden"></p>

<select id="filter">
  <option value="all">All</option>
  <option value="pass">Pass</option>
  <option value="fail">Fail</option>
</select>

<table border="1">
  <thead><tr><th>Name</th><th>Marks</th><th>Status</th></tr></thead>
  <tbody id="tbody"></tbody>
</table>
<p id="summary">Total: 0 | Pass: 0 | Fail: 0</p>
```

GIVEN CSS

```
.pass { color: green; }
.fail { color: red; }
.hidden { display: none; }
#error { color: #c0392b; }
```

Task A - HTML / CSS tweak

Add **10px** padding to all table cells and make the table **width: 100%** with **border-collapse: collapse**.

Task B - JavaScript

- Clicking Add with a valid name and marks appends a row: name, marks, and the status "pass" (marks ≥ 40) or "fail". The row gets the class **pass** or **fail**. Then both inputs are cleared.
- Validation: a blank name (after trimming), a blank marks field, or marks outside 0-100 must NOT add a row. Show "Enter a valid name and marks between 0 and 100" in **#error** instead (no **alert**). Hide **#error** after a valid add.
- The dropdown shows all rows, only pass rows, or only fail rows.
- Edge case: if the filter is set to "Fail" and a passing student is added, that new row must be hidden straight away.
- #summary** always reads **Total: N | Pass: P | Fail: F** for ALL students, regardless of the current filter.

LEVEL 4 - CHALLENGE

Questions 16 - 19 | about 140 minutes in total

Persistence, sorting, multi-rule validation and features that interact with each other. Attempt these once Level 3 feels comfortable.

Q16 To-Do List with localStorage and Filters

Hard
Target time: 30 min

Concepts: localStorage + JSON, rendering from an array, filter views, index bugs

A to-do app that remembers its tasks after a page reload and can show all, active or completed tasks.

GIVEN HTML

```
<div class="input-row">
  <input type="text" id="task-input" placeholder="Enter task...">
  <button id="add-btn">Add Task</button>
</div>
<div class="filters">
  <button class="filter-btn active" data-filter="all">All</button>
  <button class="filter-btn" data-filter="active">Active</button>
  <button class="filter-btn" data-filter="completed">Completed</button>
</div>
<ul id="task-list"></ul>
<p><span id="remaining">0</span> task(s) remaining</p>
<button id="clear-done-btn">Clear Completed</button>
```

GIVEN CSS

```
ul { list-style: none; padding: 0; }
li { padding: 6px; cursor: pointer; border-bottom: 1px solid #334155; }
li.completed { text-decoration: line-through; color: #64748b; }
.filter-btn.active { background: #2c3e50; color: #fff; }
```

Task A - HTML / CSS tweak

Add a **#94a3b8** border and **6px** border-radius to the filter buttons.

Task B - JavaScript

- Keep the tasks in an array of objects (**{ text, done }**) and save it to **localStorage** under the key **tasks** as JSON after every change.
- On page load, restore the tasks and their completed state. Missing or empty storage must not crash the page.
- Write one **render()** function that rebuilds **#task-list** from the array, using the currently selected filter (all / active / completed). Completed tasks get the class **completed**.
- Clicking a task toggles its completed state. This must affect the correct task even when the Active or Completed view is showing.
- The filter buttons switch the view and move the **active** class. The selected filter does not need to survive a reload.
- **#remaining** shows the number of tasks not yet completed across ALL tasks, not just the visible ones.
- **#clear-done-btn** removes every completed task and saves the result.
- Do not use inline **onclick** attributes. Use event listeners.

Q17 Cart with Quantity Controls and Coupons

Hard
Target time: 35 min

Concepts: delegation, closest(), recalculation, conditional discounts, empty state

A checkout cart where quantities can be changed, rows removed, and a coupon code applied. Every number on the page must stay correct after each action.

GIVEN HTML

```
<table id="cart">
  <thead><tr><th>Item</th><th>Price</th><th>Qty</th><th>Subtotal</th><th></th></tr></thead>
  <tbody>
    <tr data-price="40">
      <td>Headphones</td><td>$40</td>
      <td><button class="minus">-</button> <span class="qty">1</span>
        <button class="plus">+</button></td>
      <td class="line-total">$40.00</td><td><button class="remove">Remove</button></td>
    </tr>
    <tr data-price="25">
      <td>Keyboard</td><td>$25</td>
      <td><button class="minus">-</button> <span class="qty">1</span>
        <button class="plus">+</button></td>
      <td class="line-total">$25.00</td><td><button class="remove">Remove</button></td>
    </tr>
    <tr data-price="10">
      <td>Mouse Pad</td><td>$10</td>
      <td><button class="minus">-</button> <span class="qty">1</span>
        <button class="plus">+</button></td>
      <td class="line-total">$10.00</td><td><button class="remove">Remove</button></td>
    </tr>
  </tbody>
</table>
<p id="empty-msg" class="hidden">Your cart is empty.</p>

<input type="text" id="coupon" placeholder="Coupon code">
<button id="apply-btn">Apply</button>
<p id="coupon-msg"></p>

<p>Subtotal: $<span id="subtotal">0.00</span></p>
<p>Discount: -$<span id="discount">0.00</span></p>
<h3>Grand total: $<span id="grand-total">0.00</span></h3>
```

GIVEN CSS

```
table { border-collapse: collapse; width: 100%; }
td, th { border: 1px solid #ccc; padding: 8px 12px; text-align: left; }
.hidden { display: none; }
#coupon-msg.ok { color: #1e8449; }
#coupon-msg.bad { color: #c0392b; }
```

Task A - HTML / CSS tweak

Style the Remove buttons red (#c0392b, white text) and make #grand-total bold and larger.

Task B - JavaScript

- On page load, calculate every .line-total, #subtotal and #grand-total from the row data-price and quantity (all with two decimals).
- Use one delegated listener on the table body. + adds 1 to the quantity; - subtracts 1 but never goes below 1; Remove deletes the whole row.
- A row's .line-total is price x quantity. #subtotal is the sum of all rows.
- Coupons (case-insensitive, trimmed): SAVE10 = 10% off the subtotal. FLAT20 = \$20 off, but only when the subtotal is at least \$50, otherwise #coupon-msg says "Minimum order \$50 for FLAT20" (class bad) and no discount applies. Anything else says "Invalid coupon" (class bad). A valid coupon shows "Coupon applied: CODE" (class ok).
- The coupon stays applied when the cart changes: the discount is recalculated after every +, - and Remove. If the subtotal falls below \$50 while FLAT20 is applied, the discount becomes 0 and the minimum-order message appears.
- #grand-total = subtotal - discount and is never negative.
- When the last row is removed, show #empty-msg and set every amount back to 0.00.

Q18 Employee Table: Search, Filter and Sort

Hard
Target time: 35 min

Concepts: combined filters, numeric vs text sorting, toggling sort direction, visible totals

An employee table can be searched, filtered by department and sorted by clicking column headers. All three features must work together.

GIVEN HTML

```
<input type="text" id="search" placeholder="Search by name...">
<select id="dept-filter">
  <option value="all">All departments</option>
  <option value="Engineering">Engineering</option>
  <option value="Design">Design</option>
  <option value="Sales">Sales</option>
</select>

<table id="emp-table">
  <thead>
    <tr>
      <th id="th-name" data-sort="name">Name</th>
      <th>Department</th>
      <th id="th-salary" data-sort="salary">Salary</th>
    </tr>
  </thead>
  <tbody id="emp-body">
    <tr><td>Riya Sharma</td><td>Engineering</td><td>90000</td></tr>
    <tr><td>Aman Verma</td><td>Design</td><td>65000</td></tr>
    <tr><td>Neha Gupta</td><td>Sales</td><td>48000</td></tr>
    <tr><td>Karan Mehta</td><td>Engineering</td><td>120000</td></tr>
    <tr><td>Sara Khan</td><td>Design</td><td>72000</td></tr>
    <tr><td>Vikram Rao</td><td>Sales</td><td>58000</td></tr>
    <tr><td>Tara Iyer</td><td>Design</td><td>9500</td></tr>
  </tbody>
</table>
<p id="count"></p>
<p id="total"></p>
<p id="no-data" class="hidden">No matching employees.</p>
```

GIVEN CSS

```
table { border-collapse: collapse; width: 100%; }
td, th { border: 1px solid #ccc; padding: 8px 12px; text-align: left; }
th[data-sort] { cursor: pointer; background: #f1f5f9; }
th.asc::after { content: " (asc)"; }
th.desc::after { content: " (desc)"; }
.hidden { display: none; }
```

Task A - HTML / CSS tweak

Give even rows of the table body a light grey background using `tr:nth-child(even)`.

Task B - JavaScript

- The search box (case-insensitive, matches part of the name) and the department dropdown work together and update live.
- `#count` shows **Showing X of 7 employees**. `#total` shows **Total salary (visible): N** for the rows currently visible.
- When no rows match, show `#no-data`; otherwise hide it.
- Clicking the Name header sorts alphabetically; clicking the Salary header sorts NUMERICALLY (9500 must come before 48000). The first click sorts ascending, the next click on the same header switches to descending, and so on.
- The sorted header gets the class `asc` or `desc`. Only one header shows an indicator at a time.
- Sorting must respect the filters: rows hidden by the search or department filter stay hidden after sorting, and filtering after sorting keeps the sorted order.

Q19 Registration Form with Real-Time Validation

Hard
Target time: 40 min

Concepts: input / blur / change events, per-field rules, touched state, enabling submit

A registration form validates each field as the user types. The Register button becomes clickable only when the whole form is valid.

GIVEN HTML

```
<form id="reg-form" novalidate>
  <div class="field">
    <label for="fullname">Full name</label> <input type="text" id="fullname">
    <small class="err" id="err-fullname"></small>
  </div>
  <div class="field">
    <label for="email">Email</label> <input type="text" id="email">
    <small class="err" id="err-email"></small>
  </div>
  <div class="field">
    <label for="phone">Phone</label> <input type="text" id="phone">
    <small class="err" id="err-phone"></small>
  </div>
  <div class="field">
    <label for="password">Password</label> <input type="password" id="password">
    <small class="err" id="err-password"></small>
  </div>
  <div class="field">
    <label for="confirm">Confirm password</label> <input type="password" id="confirm">
    <small class="err" id="err-confirm"></small>
  </div>
  <div class="field">
    <label><input type="checkbox" id="terms"> I accept the terms</label>
    <small class="err" id="err-terms"></small>
  </div>
  <button type="submit" id="submit-btn" disabled>Register</button>
</form>
<p id="success" class="hidden"></p>
```

GIVEN CSS

```
.field { margin-bottom: 12px; }
.err { display: block; color: #c0392b; min-height: 16px; }
input.invalid { border: 2px solid #c0392b; }
input.valid { border: 2px solid #1e8449; }
button:disabled { opacity: 0.5; cursor: not-allowed; }
.hidden { display: none; }
```

Task A - HTML / CSS tweak

Make the Register button full width with a blue background (#2563eb) and white text.

Task B - JavaScript

- Rules. Full name: at least 3 characters after trimming, letters and spaces only. Email: has text before an @ and a . somewhere after it. Phone: exactly 10 digits. Password: at least 8 characters with at least one digit and one uppercase letter. Confirm: equals the password. Terms: checkbox ticked.
- Validate each text field on every **input** event (the checkbox on **change**). An invalid field gets the class **invalid** and its message in its **.err** element; a valid field gets **valid** and an empty message. Write clear, specific messages (for example, say which password rule is broken).
- Touched state: do not show an error for a field until the user has interacted with it (typed in it or left it).
- Changing the password must re-check the confirm field if it already has a value.
- #submit-btn** is enabled only when ALL six rules pass, even for fields the user has not touched yet.
- On submit: stop the default submission, hide the form and show "Welcome, <first name>! Your account has been created." in **#success**, using the first word of the full name.